Rushd-ul-Ilm

AI-Powered Islamic Knowledge Q&A; App

Complete System Design Architecture & Project Development Plan

Version 3.0 — Updated & AI-Agent Ready

Developer: Shaik Hidayatullah, Kurnool, Andhra Pradesh, India

For AI Coding Agents (Claude, Gemini-CLI, Codex, etc.): Read AGENT_RULES.md first. Then read ACTIVITY_LOG.md to find current phase and last completed task. Then read the relevant section in this document. Never hallucinate Islamic rulings. Always cite source URLs in every answer generated.

Section 1 — Project Overview & Target Audience

Rushd-ul-Ilm is an AI-powered Islamic knowledge Q&A; Android application designed specifically for Muslims in Kurnool, Andhra Pradesh, India, who face significant language and literacy barriers. The app replaces text-based interaction entirely with voice — users speak in their native language and receive spoken answers with authentic Islamic references.

Official App Name: Rushd-ul-Ilm (■■■ ■■■■■ — Growth of Knowledge)

Core Problems Solved

Problem	Affected Users	Engineering Solution
Cannot read, write, or type in English	Most Muslims in Kurnool	Voice-first interface — mic in, speaker out
Cannot find authentic fatwas with references	All users	RAG pipeline — LlamaIndex + Qdrant + Qwen3:4b
YouTube distractions and ads on Islamic lectures	All users	Offline video database — pre-downloaded lecture folder
Unreliable internet access in rural Kurnool	Most users	Three-tier fallback — internet → LAN → fully offline
Cannot read Urdu script (know spoken Urdu)	Many users	Telugu/Urdu voice I/O — Whisper STT + IndicTrans2 + Coqui TTS

Approved Islamic Knowledge Sources

■ CRITICAL FOR AI AGENTS: Every answer displayed in the app MUST include the original source URL as a clickable reference. Never allow the LLM to invent Islamic rulings. The RAG prompt must always say: 'Answer ONLY from the provided context. Cite the source URL.'

Source	URL	Madhab / Stance	Offline Copy Available
islamqa.info	https://islamqa.info/en	Neutral — no madhab	YES — fast_mirror.py script exists
islamqa.org	https://islamqa.org	Neutral — no madhab	Planned (future)
Darul Ifta Deoband	https://darulifta-deoband.com/en	Hanafi madhab	YES — deoband_offline_downloader.py exists
Custom Sources	User-configurable	Any	PDFs, Obsidian .md, offline HTML

Offline copy scripts location on developer machine:

islamqa.info: `/home/hidayat/Documents/Islamic Knowledge Q&A; App/islamqa.info-offline/`

darulifta-deoband: `/home/hidayat/Documents/Islamic Knowledge Q&A; App/darulifta-deoband.com-offline/`

Section 2 — Full System Architecture (6 Layers)

The app is built in six distinct layers. Layers 1–2 run on the Android phone. Layers 3–6 run on the self-hosted Ubuntu server (or any Linux VPS). Every layer has an offline fallback that runs on-device when no network is available.

Layer 1 — **Android App**
Kotlin + Jetpack Compose
What the user sees and touches. Voice input, source selector, answer screen, video library.

▼

Layer 2 — **AI / NLP Core**
Ubuntu Server — Docker
STT, Translation, TTS, LLM — all inference on your GTX 1650ti. No data leaves your machine.

▼

Layer 3 — **Islamic Knowledge Sources**
Scrapy / Go Colly + SQLite + Qdrant
Scraped fatwa content from approved sources. Stored in SQLite for offline download + Qdrant for semantic search.

▼

Layer 4 — **Vector DB + RAG Pipeline**
Qdrant + LlamaIndex + Qwen3:4b
Embeds queries, finds relevant fatwa chunks, generates cited answers. On-device fallback: pre-downloaded SQLite DB.

▼

Layer 5 — **Islamic Video Database**
FFmpeg + Whisper + ExoPlayer
Offline folder of Islamic lectures. Transcribed by Whisper, indexed in Qdrant, played by ExoPlayer.

▼

Layer 6 — **Self-Hosted Backend**
Docker Compose → any Linux VPS
FastAPI (port 8000) + Ollama (11434) + Qdrant (6333) + IndicTrans2 (8001) + Coqui TTS (8002).

Section 3 — End-to-End Voice Query Flow

This is the complete journey every time a Muslim user presses the microphone button and asks an Islamic question in Telugu or Urdu. All 8 steps run automatically.

Step 1 — User speaks into mic
AudioRecord captures 16kHz mono WAV | Example: '[redacted]?' (Can I make dua in Telugu during Salah?)

▼

Step 2 — Speech-to-Text (STT)
ConnectivityManager checks network → Online: faster-whisper-turbo GPU server | Offline: whisper.cpp JNI on-device → Returns Telugu/Urdu text

▼

Step 3 — IndicTrans2 Translation: Telugu → English
ai4bharat/indictrans2-indic-en-1B on Ubuntu GPU → 'Can I make dua in Telugu during Salah?' | Offline: Opus-MT ONNX INT8 on-device

▼

Step 4 — Sentence-Transformer Embedding
paraphrase-multilingual-mpnet-base-v2 converts query to a 768-dimension vector (a list of numbers representing meaning)

▼

Step 5 — Qdrant Semantic Search in Selected Source
metadata_filters select user-chosen source (islamqa.info / deoband / custom) → top 3-5 most relevant fatwa chunks returned with source URLs

▼

Step 6 — Qwen3:4b via Ollama — Cited Answer Generation
Prompt: 'Answer ONLY from the provided context. Cite the source URL.' → NO hallucination — cannot invent Islamic rulings outside retrieved text

▼

Step 7 — IndicTrans2 Translation: English → Telugu
Answer translated back to user's language. Source URL is preserved and appended as a clickable reference below the answer text

▼

Step 8 — Answer Displayed + Read Aloud
Coqui XTTS-v2 (online) or Android TextToSpeech (offline) reads answer in Telugu | Relevant video lectures shown below with source reference

Section 4 — Layer 1: Android App (Kotlin + Jetpack Compose)

Jetpack Compose is Google's modern UI toolkit for Android. Instead of writing XML layout files, you write UI directly in Kotlin code. It is the current industry standard and the best starting point for a beginner in 2025–26 because you learn one system, not two.

Four App Screens

Screen	Key UI Elements	Jetpack Compose Components Used
Home Screen	Large mic button (centre), language selector, source selector (islamqa/deoband)	@Composable, Button, DropdownMenu, Row, Column
Answer Screen	Answer text in Telugu/Urdu, 'Read Aloud' button (large), source URL link	LazyColumn, Text, IconButton, Clickable annotation
Video Library	Searchable list of Islamic lectures, topic search bar	LazyColumn, SearchBar, VideoItem composable
Settings	Language preference, Madhab selector, offline mode toggle	Switch, RadioButton, Slider, Preference composable

Android App Tech Stack

Component	Technology	Purpose
Language	Kotlin 2.x	Primary Android development language
UI Framework	Jetpack Compose (latest BOM)	Modern declarative UI — no XML needed
Architecture	MVVM + StateFlow + Hilt DI	Standard Android architecture pattern
Local Database	Room (SQLite wrapper)	Cached Q&A; answers, video metadata, user preferences
HTTP Client	Retrofit + OkHttp	Calls FastAPI server endpoints
Video Player	ExoPlayer / Media3	Offline Islamic lecture playback
Audio Capture	AudioRecord API	16kHz mono PCM — required by Whisper
Network Check	ConnectivityManager	Routes requests to online or offline engine
Offline Translation	ONNX Runtime Android	Runs Opus-MT ONNX INT8 models on-device
Package Name	com.rushdululilm.app	Unique Android app identifier

UI/UX Rules for Illiterate Users

Critical Design Principles

- Microphone button MUST dominate the home screen — at least 40% of screen height
- All button labels in Telugu AND English (not English only)
- Use large icons — minimum 48dp touch targets
- Green color for Islamic/Halal answers, calm blue for Quranic references
- 'Read Aloud' button always visible on the answer screen — never hidden in a menu
- Offline Mode indicator — show a clear banner when internet is unavailable

- No small text anywhere — minimum font size 16sp for all user-facing text

Section 5 — Layer 2: AI / NLP Core

5A — Speech-to-Text (STT): Three-Tier System

STT converts the user's spoken Telugu or Urdu into text. Two engines are used: **faster-whisper-large-v3-turbo** on the Ubuntu GPU server (online), and **whisper.cpp via JNI** on the Android phone itself (offline).

Tier	Condition	Engine	VRAM / RAM	Latency
Tier 1 — Online	Internet available	faster-whisper-large-v3-turbo INT8	1.5GB VRAM (GPU)	~1 second
Tier 2 — LAN	Local Wi-Fi only	Same server via LAN IP (192.168.x.x:8000)	1.5GB VRAM (GPU)	~1 second
Tier 3 — Offline	No network at all	whisper.cpp JNI — ggml-small-q5_0 (on-device)	~1 GB phone RAM	~8 sec / 30s clip

Offline STT Implementation (Android NDK + JNI)

- whisper.cpp compiled as libwhisper.so using Android NDK + CMake
- Model: ggml-small-q5_0 — 182 MB on-disk, ~1 GB RAM, 99 languages including Telugu and Urdu
- Integration: whisper.android example in whisper.cpp repo — call fullTranscribe() via JNI
- AudioRecord configured at 16kHz mono PCM — no resampling needed
- Model downloaded on first Wi-Fi launch, cached in context.filesDir — then fully offline
- Show 'Offline Mode — processing may take a few seconds' indicator in the UI

5B — Translation: IndicTrans2 (Online) + Opus-MT ONNX (Offline)

Tier	Engine	Model	Languages	Size
Tier 1–2 (Server)	IndicTrans2	ai4bharat/indictrans2-indic-en-1B	Telugu, Urdu, Hindi, Arabic	0 MB (server-side)
Tier 3 (On-device)	Helsinki-NLP Opus-MT ONNX INT8	opus-mt-te-en + opus-mt-en-mul	Telugu ↔ English	~375 MB
Tier 3 (On-device)	Helsinki-NLP Opus-MT ONNX INT8	opus-mt-ur-en + opus-mt-en-ur	Urdu ↔ English	~150 MB

■ Offline translation accuracy caveat: Show 'Offline translation — some nuance may be reduced' when using Tier 3. Opus-MT is adequate for everyday Islamic Q&A; but less accurate than IndicTrans2 for complex fiqh terminology.

5C — Text-to-Speech (TTS): Coqui XTTS-v2 (Online) + Android TTS (Offline)

Tier	Engine	Voice Quality	Languages	Size
Tier 1 — Internet	Coqui XTTS-v2 (FastAPI server)	Best — natural, voice cloning	Telugu, Urdu, Arabic, English (30+)	0 MB (server-side)
Tier 2 — LAN Wi-Fi	Same Coqui server via local IP	Best — same model	Same as Tier 1	0 MB (server-side)
Tier 3 — Offline	Android TextToSpeech API + Google Telugu voice pack	Good — robotic but understandable	Telugu, Urdu (downloadable packs)	~50 MB voice pack

Offline TTS Setup

- Android TextToSpeech API is built into Android — no library needed
- Prompt user on first launch to download Telugu + Urdu Google voice packs
- Voice packs download once over Wi-Fi and then work completely offline forever
- Use ACTION_CHECK_TTS_DATA and ACTION_INSTALL_TTS_DATA intents for setup

5D — LLM + RAG: Ollama + Qwen3:4b + LlamaIndex + Qdrant

Component	Technology	Specs	Purpose
LLM Runtime	Ollama	Runs at localhost:11434	Serves LLM locally — like Docker but for AI models
LLM Model	Qwen3:4b Q4_K_M	2.5 GB disk, 256K context, fits 4GB VRAM	Generates cited answers from retrieved context
RAG Framework	LlamaIndex (Python)	Connects all knowledge sources	Orchestrates Qdrant search + Ollama generation
Vector DB	Qdrant	Port 6333, self-hosted Docker	Semantic search with metadata_filters for source selection
Embedding Model	paraphrase-multilingual-mpnet-base-v2	768 dimensions, multilingual	Converts queries to vectors — works for Telugu, Urdu, Arabic, English

Section 6 — Layer 3: Islamic Knowledge Sources (Updated with Offline Support)

■ UPDATED IN v3: Layer 3 now includes full offline on-device mobile support. The SQLite database is downloadable to the phone. Users can browse and search fatwas without any internet connection after the initial download.

Scraping Architecture: Server-Side

A scraping pipeline runs on the Ubuntu server periodically (weekly or monthly). It downloads all Q&A; content from approved Islamic websites into a local SQLite database AND into Qdrant for semantic search. The SQLite database is then made available for users to download directly from the app for fully offline access.

Scraping Tools Decision

Language	Tool	Best For	Recommendation
Python	Scrapy + BeautifulSoup4	Rapid prototyping, existing scripts (fast_mirror.py)	USE THIS FIRST — scripts already exist for islamqa.info and deoband
Python	Playwright	JavaScript-heavy pages (SPAs)	Use when Scrapy cannot render JS content
Go	Colly	High-speed parallel crawling	Use for future sources when performance matters
Go	Chromedp / Rod	Browser automation in Go	Use for JS-heavy sites when Go is chosen
Rust	Scraper	Maximum performance, compiled binary	Use for production-grade long-running crawlers
Rust	Playwright-Rust / headless-chrome	Browser automation in Rust	Use for complex SPAs in production

✓ Existing scripts: fast_mirror.py and dump_to_db.py for islamqa.info, and deoband_offline_downloader.py for darulifta-deoband.com are already built. Read the README.md files in each offline folder before writing new scraping code.

Three-Tier Offline Support for Layer 3

Tier	Condition	How Q&A; Content is Accessed	Storage
Tier 1 — Online	Internet available	LlamaIndex queries Qdrant vector DB live on server. Full semantic search.	Qdrant on Ubuntu server
Tier 2 — LAN Wi-Fi	Same local network as server	Same Qdrant on server accessed via LAN IP (192.168.x.x:6333)	Qdrant on Ubuntu server
Tier 3 — Offline	No network at all	Pre-downloaded SQLite DB on phone. Keyword search via Room + FTS5 (Full-Text Search). No semantic search — keyword only.	SQLite DB on phone (~500MB for islamqa.info)

On-Device Offline Knowledge DB — Android Implementation

How the Offline Knowledge Base Works on Android

- The server generates a SQLite .db file from the scraped fatwa content
- The app shows a 'Download Islamic Knowledge Database' option in Settings

• User downloads the .db file once over Wi-Fi (islamqa.info ~500MB, deoband ~200MB)

• Room database on Android uses FTS5 (Full Text Search) for fast keyword search offline

• SQLite WAL mode enabled for fast concurrent reads without locking

• Download progress shown to user — large files need clear progress indicators

• User can choose which source databases to download (saves storage space)

• Weekly automated update check — notifies user when a newer DB is available

```
// Room FTS4/FTS5 entity for offline fatwa search on Android @Fts4 @Entity(tableName = 'fatwas_fts') data class FatwaFts( @PrimaryKey @ColumnInfo(name = 'rowid') val id: Int = 0, val question: String, val answer: String, val source_url: String, val source_name: String ) // Search query using FTS @Dao interface FatwaDao { @Query("SELECT * FROM fatwas_fts WHERE fatwas_fts MATCH :query LIMIT 10") suspend fun search(query: String): List<FatwaFts> }
```

Section 7 — Layer 4: Vector DB + RAG Pipeline (Updated with Offline Support)

■ UPDATED IN v3: Layer 4 now includes a full offline on-device fallback. When no server is available, the app uses the pre-downloaded SQLite FTS5 database for keyword search + a cached compact embedding index for basic semantic matching.

How RAG Works (Simple Explanation)

RAG (Retrieval-Augmented Generation) means the AI does NOT rely on its own memory to answer Islamic questions. Instead, it first SEARCHES your approved sources for relevant text, then uses that text to generate an answer. This prevents the LLM from inventing Islamic rulings — it can only use what was retrieved from the source.

Query Embedding	User's English question → sentence-transformer → 768-dim vector
Qdrant Semantic Search	Vector compared to all stored fatwa vectors → top 3-5 most similar chunks returned
Source Filter Applied	metadata_filters in Qdrant restrict results to user-selected source (islamqa/deoband/custom)
Context Sent to LLM	Retrieved chunks + source URLs sent to Qwen3:4b via Ollama
Cited Answer Generated	LLM answers ONLY using retrieved text. Prompt enforces: 'Cite the source URL.'
Answer Returned	LlamaIndex returns answer + source URLs to FastAPI → Android app

Layer 4 Three-Tier Offline Fallback

Tier	Condition	Search Method	Answer Generation
Tier 1–2 (Server)	Server reachable	Qdrant semantic vector search — finds meaning, not just keywords	Qwen3:4b via Ollama — cited answer with source URL
Tier 3 (Offline)	No network	Room FTS5 keyword search on pre-downloaded SQLite DB — finds exact words	Pre-cached answer text displayed directly from SQLite (no LLM generation needed offline)

Offline RAG Strategy

- During each online session, the top 500 most-searched Q&A; answers are cached in Room DB
- Background WorkManager job syncs new answers to Room DB when Wi-Fi is available
- Offline search uses Room FTS5 — fast full-text keyword search across cached answers
- If query matches nothing in FTS5, show: 'Connect to internet for more results'
- Each cached answer stores: question, answer, source_url, source_name, cached_at timestamp
- Users can also manually trigger a cache refresh from the Settings screen

Section 8 — Layer 5: Islamic Video Lecture Database (Updated with Offline Support)

■ UPDATED IN v3: Layer 5 now has a complete three-tier offline system. Videos stored on-device are always playable. The video metadata SQLite DB is downloadable to the phone. Whisper transcripts enable semantic topic search.

Four-Step Video Database Setup

Step 1 — **Index Video Metadata**
Python + FFmpeg extracts title, duration, scholar name, file path from each video file → stored in SQLite on Ubuntu server. This SQLite .db is also made downloadable to the phone.

Step 2 — **Transcribe for Search**
faster-whisper-large-v3-turbo transcribes each video file offline on Ubuntu GPU. Transcripts stored in Qdrant with video_id metadata. Enables topic search even if video title is vague.

Step 3 — **Android Integration**
ExoPlayer / Media3 plays videos from local phone storage (if downloaded) or from server (if online). SQLite metadata DB on phone enables offline video browsing.

Step 4 — **Smart Recommendation**
When user gets a fatwa answer, the app also searches video Qdrant DB for related lectures and shows 2-3 relevant video suggestions below the answer with one-tap playback.

Layer 5 Three-Tier Offline Fallback

Tier	Condition	Video Search	Video Playback
Tier 1–2 (Server)	Server reachable	Qdrant semantic transcript search — find videos by topic spoken in the lecture	ExoPlayer streams from server OR plays locally if already downloaded
Tier 3 (Offline)	No network	Room FTS5 search on downloaded video metadata SQLite DB — keyword search on titles and scholar names	ExoPlayer plays from local phone storage — videos already downloaded

Offline Video Strategy

- Video metadata SQLite DB downloadable from app Settings (shows title, duration, scholar, topic tags)
- Users can selectively download individual video files over Wi-Fi for offline playback
- A 'Download for Offline' button on each video card — shows download progress
- Downloaded videos stored in app-specific external storage (accessible without root)
- ExoPlayer automatically uses local file if available, falls back to server URL if not
- Estimated storage: ~500MB per 10 hours of lectures at 720p compressed

Section 9 — Complete Three-Tier Offline Fallback System (All Layers)

Every component of the app has a three-tier fallback. **Tier 1** uses the cloud or VPS server over the internet. **Tier 2** uses your Ubuntu machine on the same local Wi-Fi network (home or masjid). **Tier 3** runs entirely on the phone with no network at all. The app switches tiers automatically — no user action needed.

Feature	Tier 1 — Internet (Cloud / VPS)	Tier 2 — LAN Only (Home / Masjid Wi-Fi)	Tier 3 — Fully Offline (No Network)
STT	faster-whisper-large-v3-turbo INT8 GPU | ~1 second latency	Same server via 192.168.x.x:8000 | ~1 second latency	whisper.cpp JNI — ggml-small-q5_0 | ~8 sec/30s clip on phone CPU
Translation	IndicTrans2 on Ubuntu GPU | Best accuracy for Telugu/Urdu	Same IndicTrans2 via LAN IP | Best accuracy	Opus-MT ONNX INT8 on-device | ~525 MB total | Good for basic Q&A;
TTS	Coqui XTTS-v2 | Natural voice, 30+ languages	Same Coqui XTTS-v2 via LAN IP | Same quality	Android TextToSpeech API + Google Telugu voice pack | Robotic but clear
Q&A; (RAG)	LlamaIndex + Qdrant semantic search + Qwen3:4b generation	Same server via LAN IP | Same quality	Room FTS5 keyword search on pre-downloaded SQLite DB | Cached answers only
Knowledge Sources	Full Qdrant vector DB — all sources, semantic search	Same Qdrant via LAN | Same quality	Pre-downloaded SQLite DB per source | User selects which DBs to download
Video Search	Qdrant transcript semantic search — find by topic	Same Qdrant via LAN | Same quality	Room FTS5 on downloaded video metadata SQLite | Keyword search only
Video Playback	ExoPlayer streams from server	ExoPlayer streams from LAN server	ExoPlayer plays from local phone storage (pre-downloaded files)

✓ Migration path: Replace 192.168.x.x with your VPS IP address. All Docker Compose files are environment-agnostic — one command deploys to Hetzner, DigitalOcean, Oracle Free Tier, AWS EC2, or any Linux VPS.

Section 10 — Layer 6: Self-Hosted Backend (Docker Compose)

All server-side services run inside Docker containers on your Ubuntu machine. **Docker Compose** is a tool that starts all services with a single command: `docker compose up -d`. When you are ready to deploy for real users, you copy the same docker-compose.yml to any Linux VPS and change one environment variable (the server IP).

Service	Docker Image	Port	GPU?	Purpose
FastAPI	python:3.11-slim	8000	Optional	Main API — /transcribe /translate /tts /query endpoints
Ollama	ollama/ollama	11434	YES (CUDA)	Serves Qwen3:4b LLM locally
Qdrant	qdrant/qdrant	6333	No	Vector semantic search for all sources and videos
IndicTrans2	custom Python 3.11	8001	YES (CUDA)	Telugu/Urdu/Hindi ↔ English translation
Coqui XTTS-v2	custom Python 3.11	8002	YES (CUDA)	Text-to-Speech in Telugu, Urdu, Arabic
faster-whisper	custom Python 3.11	8003	YES (CUDA)	GPU-accelerated speech recognition
Neo4j (Graphiti)	neo4j:5-community	7474/7687	No	Knowledge graph for Graphiti temporal memory
Mem0	custom Python 3.11	8100	No	Cross-session AI agent memory layer

Android App connects to the server

Retrofit Base URL Configuration (Kotlin)

- Development: http://192.168.1.100:8000 (your Ubuntu machine on LAN)
- Production: https://your-vps-domain.com:8000 (cloud VPS with HTTPS via Nginx)
- The Android app detects LAN vs internet using ConnectivityManager and pings the server
- 200ms socket timeout for LAN ping — if no response, fall back to offline mode
- Retrofit OkHttp interceptor handles automatic retry on network timeout

Section 11 — Knowledge Graph Memory: Graphiti + Mem0

The Islamic app uses two complementary memory systems to track project facts, user interactions, and AI agent context across sessions. Both are fully open-source and self-hosted on your Ubuntu machine.

Graphiti — Temporal Knowledge Graph

Graphiti (github.com/getzep/graphiti) is an open-source Python framework for building temporally-aware knowledge graphs. Unlike regular databases, Graphiti stores WHEN facts were true — so it knows not just 'what' but 'when'. It is stored in Neo4j and has 20,000+ GitHub stars as of 2026.

Use Case in This Project	Example
Track which Islamic sources have been indexed	islamqa.info last indexed: 2026-05-01, 45,230 Q&As; stored
Track development decisions with timestamps	Decision: Use Qwen3:4b — made 2026-05-21 — reason: fits 4GB VRAM
Track which video files have been transcribed	video_id_1234.mp4 transcribed: 2026-05-10, 1,247 tokens
Track user query patterns (privacy-safe)	Topic 'salah timing' queried 47 times — suggest caching more answers
Track AI agent session handoffs	Claude session ended Phase 2, Gemini-CLI resumed Phase 3

Mem0 — Cross-Session AI Agent Memory

Mem0 (open-source, self-hosted) is a memory layer that sits between your AI agent and the LLM. It automatically extracts key facts from conversations and retrieves them in future sessions. Works with Ollama locally — no OpenAI key needed.

Memory Type	What It Stores	Example
User memory	Developer preferences persisted across all sessions	Developer prefers Kotlin explanations with analogies. Always explain JNI before using it.
Session memory	Context within a single coding session	Currently building SttManager.kt in Phase 1. Last error was missing NDK path.
Agent memory	Facts tied to a specific AI agent instance	Claude agent: last worked on Phase 2 backend setup. Ollama was running correctly.

How to Use Graphiti + Mem0 to Resume After Context Loss

- Mem0 automatically stores the last task completed and current phase in memory
- A new AI agent retrieves Mem0 memories at session start — instant context recovery
- Graphiti stores the complete project knowledge graph — all files, decisions, relationships
- Combined: new agent reads Mem0 (what was last done) + Graphiti (full project graph)
- This replaces the need to paste the entire PDF to every new AI agent session
- Setup: docker compose includes Neo4j + Graphiti + Mem0 — starts with one command

Section 12 — Complete Technology Stack Reference

Android App

Component	Technology	Specific Tool / Model	Category
Language	Kotlin	Kotlin 2.x	On-device
UI Framework	Jetpack Compose	Compose BOM latest	On-device
Architecture	MVVM + StateFlow	ViewModel + Hilt DI	On-device
Local DB	Room (SQLite)	Cached Q&A; + video metadata + downloaded fatwa DBs	On-device
HTTP Client	Retrofit + OkHttp	Calls FastAPI server	On-device
Video Player	ExoPlayer / Media3	Offline lecture playback	On-device
Audio Capture	AudioRecord API	16kHz mono PCM	On-device
Network Check	ConnectivityManager	Routes online / offline decisions	On-device
Offline Translation	ONNX Runtime Android 1.17.3	Runs Opus-MT ONNX INT8	On-device
Offline STT	whisper.cpp via Android NDK + JNI	ggml-small-q5_0 (182MB)	On-device
Offline TTS	Android TextToSpeech API	Google Telugu + Urdu voice packs	On-device

Server-Side (Ubuntu → any Linux VPS)

Component	Technology	Specific Tool / Model	Port
API Server	FastAPI (Python 3.11)	All endpoints: /transcribe /translate /tts /query	8000
LLM Runtime	Ollama	Serves Qwen3:4b locally	11434
LLM Model	Qwen3:4b Q4_K_M	2.5GB disk, 256K context, fits 4GB VRAM	via Ollama
RAG Framework	LlamaIndex	Connects Qdrant + Ollama + knowledge sources	Internal
Vector DB	Qdrant	Self-hosted, metadata_filters for source selection	6333
Embedding Model	sentence-transformers	paraphrase-multilingual-mpnet-base-v2	Internal
Online STT	faster-whisper	large-v3-turbo INT8 (1.5GB VRAM)	8003
Online Translation	IndicTrans2	ai4bharat/indictrans2-indic-en-1B (GPU)	8001
Online TTS	Coqui XTTS-v2	tts_models/multilingual/multi-dataset/xtts_v2	8002
Knowledge Graph	Graphiti + Neo4j	Temporal graph memory — github.com/getzep/graphiti	7474/7687
Agent Memory	Mem0 (self-hosted)	Cross-session AI agent memory + Ollama local	8100
Containerization	Docker + Docker Compose	One file deploys all services	varies

Data Ingestion Pipeline

Task	Tool	Output
Web scraping (Python)	Scrapy + BeautifulSoup4 + Playwright (JS sites)	SQLite DB per source + JSON for Qdrant ingestion
Web scraping (Go — future)	Colly + Chromedp/Rod	High-speed parallel crawling for new sources
PDF extraction	PyMuPDF	Text chunks for Qdrant embedding
Markdown / HTML	Python built-in + BeautifulSoup4	Text chunks from Obsidian notes, offline HTML
Video transcription	faster-whisper-large-v3-turbo	Searchable transcripts stored in Qdrant
Embedding	sentence-transformers	paraphrase-multilingual-mpnet-base-v2
Offline DB export	SQLite dump + gzip compress	Downloadable .db.gz file for Android app users

Section 13 — Six-Phase Build Order (Learning Path)

Build in this exact sequence. Each phase produces something **working** before moving to the next — you will never have a 'blank app' feeling. Total estimated time: 3–4 months at 2–3 hours per day alongside learning.

Phase 1 — Android UI Skeleton

Est. 2–3 weeks

TASK:	Set up Android Studio → Kotlin + Jetpack Compose project → build 4 screens with placeholder (fake) data
OUTPUT:	Working app with big mic button, language selector, answer screen (fake text), video list (empty)
NOTES:	*Package: com.rushdululilm.app | Use Hilt DI from day one | Add string resources in Telugu + English*
DOC:	`Report Documentation/02_ANDROID_APP_LAYER.md`

▼

Phase 2 — Backend Services on Ubuntu

Est. 1–2 weeks

TASK:	Docker Compose → FastAPI + Ollama (qwen3:4b) + Qdrant → test all endpoints via curl
OUTPUT:	LLM answering Islamic questions in English at localhost:8000/query with source references
NOTES:	*Install NVIDIA container toolkit for Docker GPU access | Test with: ollama run qwen3:4b*
DOC:	`Report Documentation/09_BACKEND_DOCKER.md`

▼

Phase 3 — Knowledge Ingestion Pipeline

Est. 2–3 weeks

TASK:	Use existing fast_mirror.py + dump_to_db.py scripts for islamqa.info → embed into Qdrant → LlamaIndex RAG
OUTPUT:	LlamaIndex RAG pipeline answering fatwa questions with source URL citations in English
NOTES:	*Read README.md in islamqa.info-offline/ before writing any new code | Also ingest deoband data*
DOC:	`Report Documentation/07_KNOWLEDGE_SOURCES.md + 06_RAG_PIPELINE.md`

▼

Phase 4 — Connect Android App to Backend

Est. 2–3 weeks

TASK:	Retrofit wires mic button → /transcribe endpoint → answer screen shows real RAG results from server
OUTPUT:	Working voice Q&A; in English with real Islamic source references shown in the app

NOTES:	*Implement three-tier network check in SttManager.kt | Add Retrofit API service interface*
DOC:	`Report Documentation/02_ANDROID_APP_LAYER.md + 03_STT_PIPELINE.md`

▼

Phase 5 — Multilingual Layers + Offline Models

Est. 3–4 weeks

TASK:	IndicTrans2 server translation → Coqui TTS → whisper.cpp JNI offline STT → Opus-MT offline translation → Android TTS
OUTPUT:	Full voice Q&A; in Telugu and Urdu with read-aloud. Offline mode fully working on phone.
NOTES:	*Build JNI bridge for whisper.cpp first | Download Opus-MT ONNX models from HuggingFace using Optimum*
DOC:	`Report Documentation/03_STT_PIPELINE.md + 04_TRANSLATION_PIPELINE.md + 05_TTS_PIPELINE.md`

▼

Phase 6 — Video Library + Offline DBs + Deployment

Est. 2–3 weeks

TASK:	Whisper transcribes video folder → Qdrant indexes → ExoPlayer plays → SQLite DBs downloadable → Docker migrated to VPS
OUTPUT:	Complete app with searchable video library, downloadable offline knowledge DBs, deployed for Kurnool users
NOTES:	*Set up Nginx + Let's Encrypt HTTPS on VPS | Generate compressed SQLite exports for user download*
DOC:	`Report Documentation/08_VIDEO_DATABASE.md + 10_OFFLINE_STRATEGY.md + 09_BACKEND_DOCKER.md`

Section 14 — AI Agent Context Files & Context Memory Solutions

When an AI coding agent (Claude, Gemini-CLI, Codex, Cursor) runs out of context tokens or you switch providers, the new agent has no memory of what was built. These files and tools solve that problem permanently.

Required Project Files (Create These First)

File	Location	Purpose	Who reads it
AGENT_RULES.md	Project root	Complete project rules, tech stack, Islamic source rules, beginner instructions	Every AI agent — FIRST file to read
ACTIVITY_LOG.md	activity-logs/	Session-by-session log — what was built, current phase, what to do next	Every AI agent — tells it where to resume
REPORT_DOC_RULES.md	Project root	Rules for maintaining Report Documentation folder	Every AI agent — tells it how to document
Report Documentation/	Project root folder	13 detailed documentation files — one per major component	AI agents resuming from specific components
knowledge-graph/GRAPHITI_SETUP.md	knowledge-graph/	Graphiti installation, Neo4j setup, usage guide	AI agent setting up memory layer
knowledge-graph/MEM0_SETUP.md	knowledge-graph/	Mem0 self-hosted setup, Ollama integration guide	AI agent setting up cross-session memory

Open-Source Context Memory Solutions (Self-Hosted)

Tool	What It Does	Self-Hosted?	Works with Ollama?	Best For
Graphiti (getzep/graphiti)	Temporal knowledge graph — stores WHEN facts were true, not just what. Neo4j-backed.	YES — Neo4j + Python	YES	Project knowledge, architectural decisions, file relationships, timeline tracking
Mem0 (open-source)	Extracts key facts from conversations. Retrieves relevant memories for new sessions.	YES — PostgreSQL + pgvector	YES — uses local Ollama for extraction	Developer preferences, session summaries, agent handoffs between Claude/Gemini/Codex
MemOS (MemTensor/MemOS)	Full memory OS — SQLite local, hybrid FTS5 + vector search, multi-agent memory sharing.	YES — SQLite local plugin	YES	100% on-device memory, no cloud dependencies, multi-agent task collaboration
Letta (formerly MemGPT)	Three-tier memory: core (always in-context), archival (vector store), recall (history).	YES — self-hosted server	YES	Long-running coding agents that need large persistent memory across weeks

How to Resume with a New AI Agent (Step-by-Step)

- 1. Give the new agent this PDF and the AGENT_RULES.md file
- 2. Agent reads AGENT_RULES.md — understands complete project, tech stack, rules
- 3. Agent reads ACTIVITY_LOG.md — finds the last entry, reads NEXT_TASK field
- 4. Agent reads the specific Report Documentation file for the current phase
- 5. If Mem0 is running: agent queries Mem0 for developer preferences and last session facts
- 6. If Graphiti is running: agent queries Neo4j for current project knowledge graph state
- 7. Agent continues from NEXT_TASK — NO need to explain the project from scratch
- 8. Agent appends a new entry to ACTIVITY_LOG.md when done and updates docs

✓ **Goal: Any AI agent (Claude, Gemini-CLI, GitHub Copilot, Codex, Cursor) can pick up this project cold and know exactly where to continue — in under 5 minutes of reading.**

Section 15 — Privacy, Security & Cloud Migration

Privacy Guarantees

What NEVER leaves your infrastructure

- User voice recordings — processed by Whisper on your machine or on-device
- Islamic questions asked by users — never sent to Google, OpenAI, or any ad network
- User query history — stored only in your local Qdrant + SQLite
- All LLM inference — Qwen3:4b via Ollama on your GPU, no cloud LLM APIs
- Translation — IndicTrans2 on your GPU, Opus-MT on the phone — never leaves device
- Ollama, Qdrant, FastAPI, Whisper, IndicTrans2, Coqui — all fully open-source with zero telemetry

Migration Path: Ubuntu Laptop → Any Linux VPS

Step	Action	Command
1	Copy docker-compose.yml to VPS	scp docker-compose.yml user@vps-ip:~/rushd-ul-ilm/
2	Set up Nginx + HTTPS on VPS	sudo apt install nginx certbot python3-certbot-nginx
3	Change Android app BASE_URL	Change 192.168.1.100 to your VPS domain in Retrofit config
4	Export Qdrant data from local machine	docker run qdrant/qdrant-backup export ./qdrant-backup/
5	Import Qdrant data on VPS	docker run qdrant/qdrant-restore import ./qdrant-backup/
6	Copy SQLite databases	scp *.db user@vps-ip:~/rushd-ul-ilm/data/
7	Start all services on VPS	docker compose up -d
8	Test all endpoints	curl https://your-domain.com/query (should return fatwa answer)

Recommended VPS Providers (Cheapest to Most Powerful)

Provider	Plan	Specs	Monthly Cost	Best For
Oracle Cloud Free Tier	Always Free Ampere	4 ARM cores, 24GB RAM	FREE	CPU-only inference — Qwen3:4b on CPU
Hetzner CX31	Shared vCPU	4 vCPU, 8GB RAM	~€7/month	Production backend for a few hundred users
Hetzner CPX41	Shared vCPU	8 vCPU, 16GB RAM	~€20/month	Full production with faster response
Hetzner GPU Server	GTX 1080	8 vCPU, 32GB RAM, 1 GPU	~€70/month	GPU inference for large user base
DigitalOcean Droplet	Basic 8GB	2 vCPU, 8GB RAM	$48/month	If you prefer DigitalOcean's UX

Quick Reference — For AI Coding Agents

Rule	What to do
FIRST ACTION	Read AGENT_RULES.md → then ACTIVITY_LOG.md → find NEXT_TASK → continue
LAST ACTION	Append a new entry to ACTIVITY_LOG.md with STATUS and NEXT_TASK
Islamic answers	ALWAYS include source URL. Prompt: 'Answer ONLY from context. Cite source URL.'
Explaining code	Explain every new concept in plain English before showing code. Developer is a beginner.
Hardware limits	GTX 1650ti = 4GB VRAM. Never suggest models needing more than 3.5GB VRAM.
Privacy rule	Never add Google Analytics, Firebase Analytics, or any third-party telemetry
New services	Every new service must be self-hostable AND have a Docker Compose entry
Offline fallback	Every feature that needs internet MUST have a three-tier fallback (internet → LAN → offline)
Package name	com.rushdululilm.app — never change this
Documentation	Update the relevant Report Documentation/ file after every code change